

Manual de Onboarding para Nuevos Desarrolladores

Santo Pegasus Soluciones

> **Versión del Documento:** 1.0.0

> **Fecha de Publicación:** Junio 2026

> **Departamento Responsable:** People & Engineering — Chapter de Back-end y Front-end

> **Clasificación:** Interno — Confidencial

> **Próxima Revisión:** Diciembre 2026

TABLA DE CONTENIDOS

1. Bienvenida Institucional y Cultura de la Empresa
2. Estructura del Equipo
3. Día 1 — Accesos y Cuentas
4. Configuración del Entorno Local — Back-end
5. Configuración del Entorno Local — Front-end
6. Guía de Git y Flujo de Trabajo
7. Primeras Tareas Sugeridas — Plan 30/60/90 Días
8. Herramientas y Sistemas Internos
9. Proceso de Code Review
10. Beneficios y Políticas de RRHH
11. Seguridad desde el Primer Día
12. Checklist de Onboarding — Semana 1
13. Contactos Útiles
14. Preguntas Frecuentes del Onboarding
15. Disposiciones Finales y Revisión del Documento

SECCIÓN 1 — BIENVENIDA INSTITUCIONAL Y CULTURA DE LA EMPRESA

1.1 Mensaje de Bienvenida

Bienvenido o bienvenida a Santo Pegasus Soluciones.

Si estás leyendo estas líneas, es porque pasaste por un proceso selectivo riguroso, demostraste tus capacidades técnicas y — lo que es igualmente importante — conectaste con los valores y la forma de trabajar de nuestro equipo. Eso no es un detalle menor. En Santo Pegasus, contratamos personas, no solo perfiles técnicos.

Este manual fue diseñado para que tu primera semana sea productiva, clara y acogedora. Nuestro objetivo es que al finalizar el séptimo día ya tengas tu entorno configurado, hayas conocido a tu equipo, comprendas nuestro flujo de trabajo y estés listo para comenzar a contribuir con código real. No te vamos a dejar solo: tendrás un buddy asignado (un desarrollador Senior o Pleno) que acompañará tus primeros 30 días.

Leé este documento con calma, seguí los pasos en orden y, ante cualquier duda, recordá: preguntar no es señal de debilidad, es señal de inteligencia.

¡Bienvenido al equipo!

— La Dirección de Ingeniería, Santo Pegasus Soluciones

1.2 Nuestra Misión

> "Construir soluciones de software que transformen negocios reales, con código que perdure, equipos que crezcan y tecnología que respete a las personas."

Santo Pegasus Soluciones es una empresa de tecnología especializada en el desarrollo de productos digitales, plataformas SaaS e integraciones complejas para clientes de diferentes industrias. Nuestra fortaleza está en la combinación de excelencia técnica con un enfoque humano: entendemos el negocio de nuestros clientes antes de escribir la primera línea de código.

1.3 Nuestros Valores

Estos no son palabras en una pared. Son los criterios reales con los que tomamos decisiones, evaluamos el trabajo y construimos la cultura del día a día:

#	Valor	Significado Práctico
1	Artesanía del Código	Escribimos código como si el próximo desarrollador que lo lea fuera alguien que sabemos que nos va a pedir explicaciones. Claridad antes que astucia.
2	Honestidad Técnica	Si algo no está bien hecho, lo decimos. Si cometemos un error, lo admitimos. No existe el "está más o menos funcionando" en producción.

3	Aprendizaje Continuo	La tecnología cambia. Nuestra curiosidad no puede parar. Invertimos en cursos, certificaciones y tiempo de estudio como parte del trabajo.
4	Colaboración sin Ego	El código no tiene dueño. Las ideas se defienden con argumentos, no con jerarquía. Un Junior puede tener razón frente a un Senior, y eso está bien.
5	Entrega con Responsabilidad	Cumplir los plazos importa. Pero cumplirlos sin comprometer la calidad importa más. Si hay un conflicto, comunicamos con anticipación.
6	Diversidad e Inclusión	Construimos mejores productos cuando nuestro equipo refleja la diversidad del mundo real. Todo tipo de discriminación es inaceptable.
7	Impacto sobre Volumen	Preferimos 10 Pull Requests bien hechos a 30 apresurados. La métrica que importa es el valor entregado, no las líneas escritas.

1.4 Cómo Trabajamos

Santo Pegasus opera en un modelo híbrido. La mayoría del equipo trabaja de manera remota, con encuentros presenciales opcionales (o requeridos en casos específicos de dinámicas de equipo o retros trimestrales). Las políticas exactas de home office se detallan en la Sección 10.

Nuestro flujo de trabajo está basado en metodología ágil (Scrum adaptado):

- Sprints de 2 semanas con planning, daily, refinement y retrospectiva.
- Daily asíncrona por Slack (canal del squad) cuando no hay reunión sincrónica agendada.
- Revisión de sprint con demostración de las funcionalidades entregadas, abierta a todos los miembros del equipo.
- Retrospectiva con espacio seguro para mejoras, frustraciones y celebraciones.

La comunicación interna ocurre principalmente por Slack. Las decisiones técnicas se documentan en Confluence. El tracking de trabajo está en Jira. El código vive en GitHub (organización privada).

1.5 Nuestra Pila Tecnológica Principal

Para que tengas el mapa completo desde el primer día:

Área	Tecnologías
------	-------------

Back-end	Java 17+, Spring Boot 3+, Spring Security, Spring Data JPA
Front-end	React 18+, TypeScript, Vite, Tailwind CSS
Bases de datos	PostgreSQL, MongoDB, Redis
Bases Vectoriales	Pinecone, Qdrant
Inteligencia Artificial	LangChain (RAG), integración con LLMs externos
Infraestructura Cloud	AWS (RDS, SES, SQS, Secrets Manager, EC2/ECS)
Contenedorización	Docker, Docker Compose
CI/CD	GitHub Actions
Observabilidad	SLF4J/Logback, Prometheus, Datadog
Calidad de Código	SonarQube, JUnit 5, Mockito, Testcontainers
Control de Versiones	Git, GitFlow, Conventional Commits

SECCIÓN 2 — ESTRUCTURA DEL EQUIPO

2.1 Organización por Chapters

Santo Pegasus organiza a sus desarrolladores en Chapters (agrupaciones por especialidad técnica) y en Squads (equipos multidisciplinarios por producto/cliente). Esta estructura permite que cada persona tenga una comunidad técnica de referencia (el Chapter) y a la vez trabaje en contextos de negocio concretos (el Squad).

...

Santo Pegasus Soluciones

Chapter de Back-end

- Tech Lead de Back-end
- Developers Senior (Back-end)
- Developers Pleno / Semi-Senior (Back-end)
- Developers Junior (Back-end)

Chapter de Front-end

- Tech Lead de Front-end
- Developers Senior (Front-end)

Developers Pleno / Semi-Senior (Front-end)

Developers Junior (Front-end)

Chapter de DevOps / SRE

Tech Lead de DevOps

Engineers Senior (DevOps)

Engineers Pleno (DevOps)

Área de People & Talent

People Business Partner

Talent Acquisition

...

2.2 Roles y Niveles

Rol	Descripción General
Junior	En formación. Trabaja con supervisión directa del buddy o Tech Lead. Foco en aprender el flujo de trabajo, las convenciones del equipo y entregar tareas bien definidas.
Pleno (Semi-Senior)	Trabaja con autonomía en features medianas. Participa activamente en Code Reviews. Capaz de resolver problemas sin supervisión constante, pero consulta en decisiones de arquitectura.
Senior	Diseña soluciones, toma decisiones técnicas, mentoriza Juniors y Plenos, lidera reviews. Referente técnico en su Chapter.
Tech Lead	Responsable técnico de un Chapter. Define estándares, valida arquitecturas, coordina con Product Managers y Tech Leads de otros Chapters. Participa en entrevistas técnicas.

2.3 Squads — División por Productos

Los squads son equipos de entrega. Cada squad tiene un Product Manager (PM), al menos un Tech Lead (puede ser compartido entre squads pequeños), desarrolladores de back-end, front-end y acceso a DevOps cuando es necesario.

Squad	Área de Negocio	Tech Stack Principal
Squad Pegasus Core	Plataforma central / API Gateway	Java + Spring Boot + PostgreSQL
Squad Phoenix	Portal del cliente / Dashboard	React 18+ + TypeScript

Squad Hermes	Integraciones y mensajería	SQS + Spring Boot + Event-Driven
Squad Athena	Módulo de IA / RAG	LangChain + Pinecone/Qdrant + Python adapter
Squad Atlas	Infraestructura y DevOps	AWS + Docker + GitHub Actions

> **Nota para nuevos integrantes:** Al incorporarte, tu Tech Lead te informará en cuál squad estarás asignado durante el onboarding y cuál será tu squad definitivo.

SECCIÓN 3 — DÍA 1 — ACCESOS Y CUENTAS

3.1 Antes de Llegar: Preparación Previa

El equipo de IT/Soporte y tu Tech Lead deben haber iniciado el proceso de creación de accesos con al menos 48 horas de anticipación a tu primer día. Si llegás al Día 1 sin correo corporativo, contactá a People (ver Sección 13).

Tu correo corporativo tendrá el formato:

...

nombre.apellido@santopegasus.com

...

3.2 Tabla Completa de Accesos — Día 1

Sistema	Para qué sirve	A quién solicitarlo	Plazo esperado
Correo corporativo (Google Workspace)	Comunicación, calendario, Drive	IT/Soporte	Antes del Día 1
Slack	Comunicación interna del equipo	IT/Soporte	Antes del Día 1
GitHub (organización privada)	Repositorios de código fuente	Tech Lead de tu Chapter	Día 1 — primeras 2h
Jira	Gestión de tareas y sprints	Tech Lead o PM del squad	Día 1
Confluence	Documentación interna	Tech Lead o PM del squad	Día 1
AWS Console (IAM User)	Acceso a servicios cloud (Read-only inicial)	Tech Lead de DevOps	Día 1–2
Datadog	Observabilidad, logs y métricas	Tech Lead de DevOps	Día 1–2

SonarQube	Análisis de calidad de código	IT/Soporte	Día 1–2
VPN Corporativa	Acceso seguro a recursos internos	IT/Soporte	Día 1
1Password (Gestor de contraseñas)	Gestión segura de credenciales	IT/Soporte	Día 1
Figma	Diseño de interfaces (Front-end)	Tech Lead de Front-end	Día 2–3

3.3 Procedimiento para Solicitar Accesos

1. Envió un correo a it-support@santopegasus.com con el asunto: **`[ONBOARDING] Solicitud de accesos — [Tu Nombre] — [Fecha de Ingreso]`**

2. En el cuerpo del correo, incluí: tu nombre completo, tu rol, el nombre de tu Tech Lead y el squad al que fuiste asignado.

3. IT/Soporte responderá con un ticket de seguimiento en un plazo máximo de 4 horas hábiles.

4. Para accesos que dependen del Tech Lead (GitHub, AWS), pedíselo directamente por Slack en el mismo Día 1.

> ⚠ Importante: Nunca compartas tus credenciales con otros miembros del equipo, ni siquiera con tu Tech Lead. Si necesitás compartir acceso temporalmente, usá roles y permisos, nunca tu usuario personal. Ver Sección 11.

3.4 Canales de Slack a los que Unirte el Día 1

Una vez en Slack, unite inmediatamente a los siguientes canales:

Canal	Propósito
`#general`	Comunicaciones generales de la empresa
`#back-end`	Discusiones técnicas del Chapter de Back-end
`#front-end`	Discusiones técnicas del Chapter de Front-end
`#devops`	Infraestructura, deploys, alertas de CI/CD
`#incidents`	Canal de incidentes en producción (solo lectura inicial)
`#code-reviews`	Notificaciones de Pull Requests abiertos
`#aprendizaje`	Recursos, cursos, artículos y certifications
`#random`	Conversaciones informales y cultura del equipo

SECCIÓN 4 — CONFIGURACIÓN DEL ENTORNO LOCAL — BACK-END

> Esta sección es para desarrolladores del Chapter de Back-end. Si sos del Chapter de Front-end, podés saltar directamente a la Sección 5.

4.1 Requisitos Previos del Sistema

Antes de comenzar, asegurate de tener:

- **Sistema operativo: macOS 13+, Ubuntu 22.04+ o Windows 11 con WSL2 (recomendado)**
- **Al menos 16 GB de RAM y 50 GB de espacio libre en disco**
- Conexión a la VPN corporativa activa

4.2 Instalación de JDK 17 con SDKMAN

Usamos **SDKMAN** para gestionar versiones del JDK. Esto permite que diferentes proyectos usen diferentes versiones de Java sin conflictos.

Paso 1 — Instalar SDKMAN:

```
```bash
curl -s "https://get.sdkman.io" | bash
source "$HOME/.sdkman/bin/sdkman-init.sh"
```
```

Paso 2 — Verificar instalación:

```
```bash
sdk version
```
```

Paso 3 — Instalar Java 17 (distribución Temurin de Eclipse):

```
```bash
sdk install java 17.0.11-tem
sdk default java 17.0.11-tem
```
```

Paso 4 — Verificar la versión activa:

```
```bash
java -version
```
```

Salida esperada: openjdk version "17.0.11"

2024-04-16

...

4.3 Instalación y Configuración de IntelliJ IDEA

Usamos IntelliJ IDEA (Community o Ultimate — la empresa provee licencias Ultimate para todos los desarrolladores back-end).

Paso 1 — Instalar IntelliJ IDEA:

Descargá desde: <https://www.jetbrains.com/idea/download/>

Paso 2 — Plugins obligatorios a instalar:

| Plugin | Propósito |
|--------------------|--|
| SonarLint | Análisis de calidad de código en tiempo real |
| Lombok | Soporte para anotaciones Lombok en Spring Boot |
| MapStruct Support | Soporte para MapStruct en conversión de DTOs |
| Docker | Gestión de contenedores desde el IDE |
| GitToolBox | Información de Git inline en el editor |
| .env files support | Soporte para archivos .env |

Paso 3 — Configuración del Code Style:

...

File → Settings → Editor → Code Style → Java
→ Importar el archivo santo-pegasus-codestyle.xml
(disponible en el repositorio interno: /docs/ide/santo-pegasus-codestyle.xml)

...

Paso 4 — Configurar el JDK en IntelliJ:

...

File → Project Structure → SDK → Add SDK → JDK
→ Apuntar a la ruta de SDKMAN: ~/.sdkman/candidates/java/current

...

4.4 Instalación de Docker Desktop

Usamos Docker obligatoriamente para levantar el ambiente local con todas las dependencias (base de datos, Redis, etc.).

```
```bash
macOS (con Homebrew)
```

```
brew install --cask docker
Ubuntu
```

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
Verificar instalación
```

```
docker --version
docker compose version
```
```

> En macOS y Windows, iniciá Docker Desktop como aplicación antes de ejecutar cualquier comando `docker`.

4.5 Clonar el Repositorio y Configurar el Proyecto

Paso 1 — Clonar el repositorio principal:

```
```bash
git clone git@github.com:santo-pegasus/[nombre-del-proyecto].git
cd [nombre-del-proyecto]
```
```

> **Necesitás tener tu SSH Key configurada en tu cuenta de GitHub. Si no la tenés:**

```
> ```bash
> ssh-keygen -t ed25519 -C "nombre.apellido@santopegasus.com"
> cat ~/.ssh/id_ed25519.pub
> # Copiá el output y agregalo en GitHub → Settings → SSH Keys
> ```
```

Paso 2 — Crear el archivo `.env` local:

```
```bash
cp .env.example .env
```
```

Paso 3 — Completar las variables obligatorias en el `.env`:

| Variable | Descripción | Ejemplo de Valor |
|----------|-------------|------------------|
|----------|-------------|------------------|

| | | |
|---------------|--|--|
| `DB_URL` | URL de conexión a PostgreSQL local | `jdbc:postgresql://localhost:5432/pegasus_dev` |
| `DB_USERNAME` | Usuario de la base de datos local | `pegasus_user` |
| `DB_PASSWORD` | Contraseña de la base de datos local | `dev_password_local` |
| `JWT_SECRET` | Secret para firma de tokens JWT (solo dev) | `my-local-dev-secret-key-256bits` |
| `AWS_REGION` | Región AWS configurada | `us-east-1` |
| `REDIS_HOST` | Host del Redis local | `localhost` |
| `REDIS_PORT` | Puerto del Redis | `6379` |
| `APP_PORT` | Puerto en el que corre la aplicación | `8080` |

> **⚠ El archivo `.env` NUNCA debe ser commiteado al repositorio. Ya está listado en el `.gitignore`. Si por error lo commiteás, notificá inmediatamente al Tech Lead. Ver Sección 11.**

4.6 Levantar el Ambiente Local con Docker Compose

```
```bash
```

**Levantar todas las dependencias (PostgreSQL, Redis, etc.)**

```
docker compose up -d
```

**Verificar que los contenedores están corriendo**

```
docker ps
```

**Salida esperada:**

**CONTAINER ID    IMAGE            STATUS            PORTS**

**abc123def456   postgres:15   Up 2 minutes  
0.0.0.0:5432->5432/tcp**

**789ghi101jkl   redis:7        Up 2 minutes  
0.0.0.0:6379->6379/tcp**

...

---

#### 4.7 Ejecutar el Proyecto por Primera Vez

```bash

Opción 1 — Con Maven Wrapper (recomendado)

./mvnw spring-boot:run

Opción 2 — Desde IntelliJ IDEA

Abrir la clase principal:

src/main/java/.../Application.java

Click derecho → Run 'Application'

Verificar que la aplicación levantó correctamente

curl http://localhost:8080/actuator/health

Respuesta esperada: {"status":"UP"}

...

4.8 Ejecutar las Pruebas

```bash

**Ejecutar todas las pruebas**

./mvnw test

**Ejecutar solo las pruebas unitarias**

./mvnw test -Dtest="UnitTest"

**Ejecutar solo las pruebas de integración (requiere Docker corriendo)**

./mvnw test -Dtest="IntegrationTest"

# Verificar el reporte de cobertura (generado en target/site/jacoco/index.html)

```
./mvnw verify
```

```
open target/site/jacoco/index.html
```

```
...
```

> **Recordá: la cobertura mínima obligatoria es del 80% en pruebas unitarias para aprobar un Code Review. [Source: Guía Oficial de Ingeniería Back-end.pdf, Página 12]**

```

```

```

```

## SECCIÓN 5 — CONFIGURACIÓN DEL ENTORNO LOCAL — FRONT-END

> Esta sección es para desarrolladores del Chapter de Front-end.

### 5.1 Requisitos Previos del Sistema

- **Sistema operativo: macOS 13+, Ubuntu 22.04+ o Windows 11 con WSL2**
- **Al menos 8 GB de RAM y 20 GB de espacio libre**
- **Conexión a la VPN corporativa activa**

```

```

### 5.2 Instalación de Node.js 20+ con NVM

Usamos NVM (Node Version Manager) para gestionar versiones de Node.js, de la misma manera que usamos SDKMAN para Java.

#### Paso 1 — Instalar NVM:

```
```bash
```

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

Recargar el shell

```
source ~/.bashrc # o ~/.zshrc en macOS con Zsh
```

```
...
```

Paso 2 — Instalar y usar Node.js 20:

```
```bash
```

```
nvm install 20
```

```
nvm use 20
```

```
nvm alias default 20
```

```
...
```

#### Paso 3 — Verificar instalación:

```
```bash
node --version  # v20.x.x
npm --version   # 10.x.x
```

```

## 5.3 Visual Studio Code — Instalación y Extensiones

Usamos VS Code como editor estándar para el Chapter de Front-end.

Descargá desde: <https://code.visualstudio.com/>

**Extensiones obligatorias:**

| Extensión                 | ID en Marketplace                            | Propósito                              |
|---------------------------|----------------------------------------------|----------------------------------------|
| ESLint                    | <code>`dbaeumer.vscode-eslint`</code>        | Linting de JavaScript/TypeScript       |
| Prettier                  | <code>`esbenp.prettier-vscode`</code>        | Formateo automático de código          |
| GitLens                   | <code>`eamodio.gitlens`</code>               | Información de Git inline en el editor |
| TypeScript Hero           | <code>`rbbbit.typescript-hero`</code>        | Organización de imports TypeScript     |
| Tailwind CSS IntelliSense | <code>`bradlc.vscode-tailwindcss`</code>     | Autocompletado para Tailwind           |
| Auto Rename Tag           | <code>`formulahendry.auto-rename-tag`</code> | Renombrado automático de tags JSX      |
| Error Lens                | <code>`usernamehd.error-lens`</code>         | Visualización inline de errores        |
| SonarLint                 | <code>`sonarsource.sonarlint-vscode`</code>  | Calidad de código en tiempo real       |

**Configuración de VS Code (settings.json):**

```
```json
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  },
  "typescript.preferences.importModuleSpecifier": "relative"
}
```

```

## 5.4 Clonar el Repositorio Front-end

```
```bash
git clone git@github.com:santo-pegasus/[nombre-del-proyecto]-frontend.git
cd [nombre-del-proyecto]-frontend
```
```

## 5.5 Configurar el Archivo `.env`

```
```bash
cp .env.example .env
```
```

**Variables obligatorias para el front-end:**

| Variable            | Descripción                  | Ejemplo                     |
|---------------------|------------------------------|-----------------------------|
| `VITE_API_BASE_URL` | URL de la API back-end local | `http://localhost:8080/api` |
| `VITE_APP_NAME`     | Nombre de la aplicación      | `Pegasus Portal`            |
| `VITE_ENVIRONMENT`  | Entorno actual               | `development`               |

> **▲ En proyectos Vite, solo las variables con prefijo `VITE\_` son expuestas al cliente. Nunca pongas secrets en variables front-end. El `.env` ya está en el `.gitignore`.**

## 5.6 Instalar Dependencias y Ejecutar el Proyecto

```
```bash
```

Instalar dependencias

```
npm install
```

Iniciar el servidor de desarrollo

```
npm run dev
```

La aplicación estará disponible en:

http://localhost:5173

Ejecutar pruebas unitarias

```
npm run test
```

Ejecutar pruebas con cobertura

```
npm run test:coverage
```

Build para producción (solo para verificar que el build no falla)

```
npm run build
```

...

SECCIÓN 6 — GUÍA DE GIT Y FLUJO DE TRABAJO

6.1 Modelo de Branching — GitFlow

Usamos GitFlow como estrategia de branching. Las ramas principales son:

Rama	Propósito	Reglas
`main`	Código en producción	Nunca se commitea directamente. Solo recibe merges desde `release/` o `hotfix/`
`develop`	Integración de features en desarrollo	Base para abrir features. Recibe merges desde `feature/`
`feature/`	Desarrollo de nuevas funcionalidades	Se crea desde `develop`. Se mergea de vuelta a `develop`
`release/`	Preparación para producción	Se crea desde `develop`. Se mergea a `main` y `develop`
`hotfix/`	Correcciones urgentes en producción	Se crea desde `main`. Se mergea a `main` y `develop`

6.2 Nomenclatura de Branches

El patrón obligatorio para nombrar una branch es:

...

[tipo]/[JIRA-TICKET]-[descripcion-corta-en-kebab-case]

...

Tipos válidos:

Tipo	Cuándo usarlo
`feature`	Nueva funcionalidad

`fix`	Corrección de bug
`hotfix`	Corrección urgente en producción
`refactor`	Refactorización de código existente
`docs`	Actualización de documentación
`test`	Adición o corrección de pruebas
`chore`	Tareas de mantenimiento (actualizaciones de deps, etc.)

Ejemplos correctos:

```

`bash
feature/PEG-142-cadastro-de-usuarios
fix/PEG-201-correccion-validacion-email
refactor/PEG-315-extraer-logica-autenticacion
hotfix/PEG-999-error-critico-login-produccion
`

```

Cómo crear la branch:

```

`bash

```

Asegurarse de estar en develop actualizado

```

git checkout develop

```

```

git pull origin develop

```

Crear y cambiar a la nueva branch

```

git checkout -b feature/PEG-142-cadastro-de-usuarios

```

```

`

```

```

---

```

6.3 Conventional Commits — Mensajes de Commit Semánticos

Usamos Conventional Commits para todos los mensajes de commit. Esto permite la generación automática de changelogs y una historia de Git legible.

Formato:

```

`

```

```

<tipo>(<scope>): <descripción corta en minúsculas>

```

```

[cuerpo opcional — explicación más detallada]

```

```

[footer opcional — referencias a tickets]

```

```

`

```

Tipos válidos:

Tipo	Significado
`feat`	Nueva funcionalidad
`fix`	Corrección de bug
`refactor`	Refactorización sin cambio de funcionalidad
`test`	Adición o corrección de pruebas
`docs`	Documentación
`style`	Formateo, espacios (sin cambios lógicos)
`chore`	Mantenimiento, actualizaciones
`perf`	Mejoras de rendimiento
`ci`	Cambios en CI/CD

Ejemplos correctos:

```
```bash
```

## Feature simple

git commit -m "feat(auth): agregar validación de token JWT en el endpoint de login"

## Fix con referencia a ticket

git commit -m "fix(usuarios): corregir validación de email duplicado

El servicio no estaba verificando correctamente los emails en mayúsculas.

Closes PEG-201"

## Refactor

git commit -m "refactor(pagos): extraer lógica de cálculo de impuestos a servicio dedicado"

```
```
```

Ejemplos incorrectos (no hacer esto):

```
```bash
```

## Demasiado vago

git commit -m "fix bug"

# Sin tipo

```
git commit -m "agrego validación"
```

## En mayúsculas

```
git commit -m "FEAT: Nueva funcionalidad"
```

```
...
```

```

```

## 6.4 Proceso de Push y Pull Request

### Paso 1 — Asegurarte de que las pruebas pasen localmente:

```
```bash
```

```
./mvnw test # Back-end
```

```
npm run test # Front-end
```

```
```
```

### Paso 2 — Push de tu branch:

```
```bash
```

```
git push origin feature/PEG-142-cadastro-de-usuarios
```

```
```
```

### Paso 3 — Abrir el Pull Request en GitHub:

- Ir al repositorio en GitHub
- **Hacer click en "Compare & pull request"**
- Completar el template obligatorio del PR:
  - **¿Qué se hizo? — Descripción clara de los cambios**
  - **¿Por qué se hizo? — Contexto de negocio o técnico**
  - **¿Cómo probar? — Pasos detallados para reproducir y verificar los cambios**
  - **Screenshots (si aplica, especialmente en front-end)**
  - **Checklist de calidad (pruebas, documentación, etc.)**

### Paso 4 — Solicitar revisores:

- Asignar al menos 2 revisores Senior o Semi-Senior (Pleno)
- Notificar en el canal `#code-reviews` de Slack con el link al PR

**> Todo PR en Santo Pegasus exige al menos 2 aprobaciones de miembros Senior o Semi-Senior (Pleno) antes del merge. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]**

```

```

## 6.5 Responder Reviews

- **Respondé todos los comentarios antes de solicitar una nueva revisión.**

- Si estás de acuerdo con un comentario, hacé el cambio y respondé con `Corregido en el commit abc1234`.
- Si no estás de acuerdo, explicá tu razonamiento con respeto y datos técnicos. El debate técnico es bienvenido.
- Nunca hagas merge sin tener todas las aprobaciones requeridas.

---

---

## SECCIÓN 7 — PRIMERAS TAREAS SUGERIDAS — PLAN 30/60/90 DÍAS

### 7.1 Para Desarrolladores Junior

| Período          | Objetivos                                                                                                                                                                                                                                           | Indicadores de Éxito                                                                             |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Primeros 30 días | Completar el onboarding. Tener el entorno funcionando. Leer la Guía de Ingeniería Back-end o Front-end completa. Resolver 2–3 tickets de tipo `good-first-issue` en Jira con apoyo del buddy. Participar en al menos 1 Code Review como observador. | PR mergeado. Entorno configurado y funcional. Participación activa en dailies.                   |
| 30–60 días       | Entregar features pequeñas de forma autónoma (con revisión del buddy). Escribir pruebas unitarias para el código propio. Aprender a leer y usar el Swagger de las APIs internas. Participar activamente en Code Reviews.                            | Cobertura de pruebas $\geq 80\%$ en código propio. Al menos 3 PRs mergeados sin errores graves.  |
| 60–90 días       | Contribuir en refinements con preguntas técnicas. Proponer mejoras en el código del equipo. Tomar ownership de al menos 1 feature completa de principio a fin. Dar su primer Code Review como revisor (con guía del buddy).                         | Feature completa entregada. Primer review dado. Participación en retros con sugerencias propias. |

---

### 7.2 Para Desarrolladores Pleno (Semi-Senior)

| Período          | Objetivos                                                                                                                                           | Indicadores de Éxito                                                                     |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Primeros 30 días | Onboarding técnico completo. Entender la arquitectura del squad. Tomar ownership de features de mediana complejidad. Realizar Code Reviews activos. | Al menos 5 PRs mergeados. Al menos 3 Code Reviews realizados con comentarios de calidad. |

|            |                                                                                                                                                                             |                                                                                        |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| 30–60 días | Liderar técnicamente una feature del sprint. Proponer mejoras de arquitectura o proceso en el squad. Mentorizar a un Junior si hay en el squad.                             | Feature liderada entregada. Propuesta técnica documentada en Confluence.               |
| 60–90 días | Tomar decisiones técnicas en su área con autonomía. Participar en definición de criterios de aceptación en refinements. Proponer y liderar una iniciativa de deuda técnica. | Decisión técnica documentada. Iniciativa de deuda técnica con PR abierto o completado. |

---

## 7.3 Para Desarrolladores Senior

| Período          | Objetivos                                                                                                                                                                                                     | Indicadores de Éxito                                                                            |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Primeros 30 días | Onboarding completo. Entender la arquitectura global (no solo del squad). Revisar la Guía de Ingeniería y proponer mejoras si las identifica. Contribuir con reviews de alta calidad desde la primera semana. | ADR (Architecture Decision Record) de una decisión tomada. Reviews reconocidos por el equipo.   |
| 30–60 días       | Identificar y documentar áreas de mejora técnica. Liderar una iniciativa técnica transversal. Comenzar a mentorizar activamente a Juniors o Plenos.                                                           | Documento técnico en Confluence. Al menos 2 Juniors/Plenos con sesión de mentoring.             |
| 60–90 días       | Participar en entrevistas técnicas de nuevos candidatos. Proponer cambios en la Guía de Ingeniería. Tener visibilidad clara sobre el roadmap técnico del squad.                                               | Participación en al menos 1 entrevista. Pull Request de actualización de la Guía de Ingeniería. |

---

---

## SECCIÓN 8 — HERRAMIENTAS Y SISTEMAS INTERNOS

### 8.1 Jira — Gestión de Tareas

URL: <https://santopegasus.atlassian.net/jira>

**Cómo rastrear tus tickets:**

1. Al inicio de cada sprint, los tickets asignados a vos aparecen en tu Board del Squad.
2. Cada ticket tiene un ID único: ej. `PEG-142`. Ese ID se usa en branches y commits.
3. Los estados de un ticket son: To Do → In Progress → In Review → Done.
4. Cuando comenzás a trabajar en un ticket, movelo a In Progress inmediatamente.
5. Cuando abrís un PR, movelo a In Review y agregá el link al PR en el ticket de Jira.
6. Solo movelo a Done cuando el PR fue mergeado.

## Tipos de tickets que vas a encontrar:

| Tipo             | Descripción                                               |
|------------------|-----------------------------------------------------------|
| Story            | Funcionalidad desde la perspectiva del usuario            |
| Task             | Tarea técnica sin valor directo de negocio                |
| Bug              | Error a corregir                                          |
| Spike            | Investigación técnica o exploración                       |
| good-first-issue | Tickets etiquetados especialmente para nuevos integrantes |

---

## 8.2 Confluence — Documentación Interna

URL: <https://santopegasus.atlassian.net/confluence>

### Espacios principales:

| Espacio                          | Qué encontrarás                                      |
|----------------------------------|------------------------------------------------------|
| Engineering / Back-end           | Guías técnicas, ADRs, arquitectura de sistemas       |
| Engineering / Front-end          | Guías de componentes, design system, convenciones    |
| Engineering / DevOps             | Procedimientos de deploy, runbooks de incidentes     |
| Squads                           | Documentación específica de cada squad y producto    |
| People & Onboarding              | Este manual, políticas de RRHH, beneficios           |
| Decisiones de Arquitectura (ADR) | Registro histórico de decisiones técnicas relevantes |

**> Buena práctica: Ante cualquier duda técnica recurrente, buscá primero en Confluence antes de preguntar en Slack. Si la respuesta no está, agregala vos mismo después de encontrarla — así mejoramos la documentación colectivamente.**

---

## 8.3 Slack — Comunicación Interna

### Canales obligatorios (ya listados en Sección 3.4) — Guía de etiqueta:

- Preferí los canales públicos a los mensajes directos para temas técnicos. Así el conocimiento queda disponible para todos.
- Usá hilos (threads) para responder a mensajes específicos. Evitá el ruido en el canal principal.
- Para menciones urgentes, usá `@nombre` o `@here` con criterio (no abusos del `@channel`).

- El canal `#incidents` tiene alertas automáticas de Datadog y PagerDuty. No envíes mensajes en este canal salvo que estés informando o coordinando un incidente activo.

---

## 8.4 Swagger — APIs Internas

**Todas las APIs de Santo Pegasus están documentadas usando la especificación OpenAPI (Swagger), generada automáticamente desde el código con `springdoc-openapi`.**

**Acceso al Swagger en ambiente local:**

...

<http://localhost:8080/swagger-ui/index.html>

...

**Acceso al Swagger en ambiente de Staging:**

...

<https://api-staging.santopegasus.com/swagger-ui/index.html>

...

> El Swagger de producción está protegido y requiere autenticación. Consultá con tu Tech Lead para obtener las credenciales de acceso al Swagger de Staging.

**Cómo usar el Swagger:**

1. Expandí el endpoint que querés explorar.
2. **Hacé click en "Try it out" para ejecutar llamadas reales.**
3. Completá los parámetros requeridos.
4. **Hacé click en "Execute" y analizá la respuesta.**

---

---

## SECCIÓN 9 — PROCESO DE CODE REVIEW

### 9.1 Filosofía del Code Review en Santo Pegasus

**El Code Review no es un filtro de control de calidad punitivo. Es una oportunidad de aprendizaje colectivo, mejora del código y diseminación de conocimiento. Todos — sin importar el nivel — dan y reciben reviews.**

> La comunicación en los reviews debe estar siempre pautada por la empatía y la educación. El enfoque de la revisión debe ser exclusivamente la mejora del código, nunca una crítica al autor. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

---

### 9.2 Cómo Participar en Reviews desde el Día 1

**Como autor del PR:**

- Completá el template del PR con detalle: qué se hizo, por qué se hizo, cómo probar.

- Hacé PRs pequeños y focalizados. Un PR que toca 15 archivos no relacionados es difícil de revisar.
- Respondé todos los comentarios antes de pedir re-revisión.
- No hagas merge sin las 2 aprobaciones requeridas.

#### Como revisor:

- Aceptá la revisión dentro de 24 horas hábiles de ser asignado.
- Leé el contexto del ticket de Jira antes de revisar el código.
- Priorizá comentarios sobre: legibilidad, seguridad, rendimiento y conformidad con la Guía de Ingeniería.

---

## 9.3 Etiqueta — Cómo Dar Feedback

#### Recomendado — Preguntas sugestivas:

...

"¿Qué te parece extraer esta lógica a un método privado para mejorar la legibilidad?"

"Noté que este método hace más de una cosa. ¿Consideraste separar las responsabilidades?"

"¿Hay alguna razón para no usar Optional aquí en lugar del null check manual?"

...

#### No recomendado — Imperativos sin contexto:

...

"Cambia esto."

"Esto está mal."

"No hagas así."

...

> Se recomienda el uso de preguntas sugestivas para incentivar la reflexión del desarrollador, como: "¿Qué te parece extraer esto a un método privado?", evitando imperativos como "Cambia esto a un método privado". [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

---

## 9.4 Criterios Técnicos de Revisión

El revisor debe validar:

| Criterio    | Qué verificar                                                                                       |
|-------------|-----------------------------------------------------------------------------------------------------|
| Legibilidad | El código es fácil de entender por alguien que no lo escribió                                       |
| Seguridad   | No hay hardcode de credenciales, validación de entradas presente, no se expone información sensible |
| Rendimiento | No hay consultas N+1, no hay operaciones bloqueantes innecesarias                                   |

|              |                                                                                      |
|--------------|--------------------------------------------------------------------------------------|
| Pruebas      | La cobertura mínima del 80% está cumplida; los casos edge están cubiertos            |
| Conformidad  | El código sigue las directrices de la Guía de Ingeniería Back-end o Front-end        |
| Arquitectura | Las responsabilidades están en las capas correctas (Controller, Service, Repository) |

> El revisor actúa como un guardián de la base de código, debiendo validar minuciosamente la legibilidad, la seguridad, el rendimiento y si la implementación respeta rigurosamente todas las directrices establecidas en la Guía de Ingeniería Back-end. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

---

---

## SECCIÓN 10 — BENEFICIOS Y POLÍTICAS DE RRHH

### 10.1 Beneficios Activos

| Beneficio                  | Detalle                                                                                                            |
|----------------------------|--------------------------------------------------------------------------------------------------------------------|
| Vale Transporte            | Cubierto al 100% para días de trabajo presencial, proporcional a los días en oficina                               |
| Vale Alimentación / Comida | Tarjeta mensual cargada el primer día hábil del mes, valor definido según nivel de seniority                       |
| Seguro de Salud            | Plan médico corporativo, cobertura desde el primer mes de trabajo                                                  |
| Seguro Dental              | Incluido en el plan de salud corporativo                                                                           |
| Horario Flexible           | Core hours de 10:00 a 17:00 (zona horaria del equipo). El resto del horario es flexible                            |
| Home Office                | Modelo híbrido. La asistencia presencial es opcional salvo eventos específicos del equipo                          |
| Cursos y Certificaciones   | Presupuesto anual por persona para capacitaciones y certificaciones técnicas. Consultá con People el monto vigente |
| Día de Cumpleaños Libre    | El día de tu cumpleaños (o el día hábil más cercano) es libre remunerado                                           |
| Política de Vacaciones     | Según legislación local + días adicionales por antigüedad. Coordinar con 30 días de anticipación                   |

---

## 10.2 Cursos y Certificaciones Patrocinados

Santo Pegasus invierte activamente en el crecimiento técnico de su equipo. Las certificaciones priorizadas son:

- **AWS Certified Developer – Associate**
- **AWS Certified Solutions Architect**
- **Oracle Certified Professional: Java SE 17 Developer**
- **Professional Scrum Developer (PSD)**
- **Kubernetes Application Developer (CKAD)**

Para solicitar el patrocinio de un curso o certificación:

1. Conversarlo con tu Tech Lead.
2. Completar el formulario en Confluence: `People → Beneficios → Solicitud de Capacitación`.
3. Aguardar aprobación del People BP (plazo: 5 días hábiles).

---

## 10.3 Política de Desconexión

Santo Pegasus respeta el tiempo fuera del trabajo. No se esperan respuestas de Slack fuera del horario laboral, salvo que seas el responsable de guardia de on-call (rotación voluntaria con compensación adicional). Los incidentes de producción tienen un proceso de escalada definido — nadie debe estar disponible 24/7 sin acuerdo previo.

---

---

# SECCIÓN 11 — SEGURIDAD DESDE EL PRIMER DÍA

## 11.1 Gestión de Contraseñas

Usamos 1Password como gestor de contraseñas corporativo. Todas las contraseñas relacionadas con sistemas de la empresa deben estar almacenadas ahí.

**Reglas básicas:**

- Nunca reutilices contraseñas entre sistemas.
- Todas las contraseñas corporativas deben tener al menos 20 caracteres, generadas por 1Password.
- **Activa autenticación de dos factores (2FA) en todos los sistemas que lo permitan: GitHub, AWS Console, Jira, Confluence.**
- **Nunca compartas tu contraseña con nadie, ni con tu Tech Lead, ni con IT/Soporte. Si necesitas dar acceso, usá roles y permisos del sistema correspondiente.**

---

## 11.2 Credenciales en Código — Regla de Oro

> **NUNCA** haga **hardcode** (escribir directamente en el código) de contraseñas, claves de API, URLs de base de datos o tokens. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

En entornos de producción y homologación, es obligatorio el uso de soluciones de gestión de secretos como HashiCorp Vault, AWS Secrets Manager o Google Cloud Secret Manager. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

### Checklist de seguridad antes de hacer un commit:

- [ ] ¿El archivo `.env` está en el `.gitignore`?
- [ ] ¿No hay ningún secret, password o API key en el código fuente?
- [ ] ¿Las variables sensibles están referenciadas por nombre de variable de entorno?
- [ ] ¿El archivo `.env.example` tiene los nombres de las variables pero sin valores reales?

---

## 11.3 Uso Correcto del `.gitignore`

El repositorio ya tiene un `.gitignore` pre-configurado. Nunca lo modifique para incluir archivos que contienen secretos. Los archivos que siempre deben estar en el `.gitignore`:

```.gitignore`

Variables de entorno locales

`.env`

`.env.local`

`.env..local`

Archivos de configuración con secrets

`application-local.yml`

`application-local.properties`

Directorios de build

`target/`

`build/`

`dist/`

`node_modules/`

Archivos de IDE

`.idea/`

`.iml`

`.vscode/settings.json`

...

11.4 Escaneo Automático de Secrets

Usamos herramientas de escaneo automático como gitleaks o trufflehog en nuestros pipelines de CI/CD para detectar y bloquear commits que contengan cadenas de texto que se asemejen a secretos. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 9]

Si el pipeline bloquea tu PR por detección de un posible secret:

1. **No intentes bypassear el control.**
2. Revisá el commit indicado en el error.
3. Si es un falso positivo, notificá al Tech Lead de DevOps.
4. **Si es un secret real, notificá inmediatamente al Tech Lead y a IT/Soporte para iniciar el proceso de rotación de credenciales.**

11.5 Reporte de Incidentes de Seguridad

Si detectás o sospechás de un incidente de seguridad (secret expuesto, acceso no autorizado, comportamiento anómalo):

1. **No lo resuelvas solo y en silencio.**
2. Notificá inmediatamente en `#incidents` con el tag `@security-lead`.
3. Contactá a tu Tech Lead directamente por Slack o teléfono.
4. Documentá lo que viste, cuándo lo viste y qué acciones tomaste.
5. No borres evidencia. No modifiques logs.

> El principio de privilegio mínimo debe ser aplicado, garantizando que cada microservicio tenga acceso solo a los secretos estrictamente necesarios para su operación. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 9]

SECCIÓN 12 — CHECKLIST DE ONBOARDING — SEMANA 1

Este checklist debe ser completado conjuntamente por el nuevo desarrollador y su Tech Lead antes del fin de la primera semana. Guardalo en Confluence en tu página personal de onboarding.

12.1 Accesos y Cuentas

- [] Correo corporativo (@santopegasus.com) funcionando
- [] Slack instalado y canales obligatorios unidos
- [] Acceso a GitHub (organización privada) confirmado

- [] Acceso a Jira funcionando, squad asignado visible
- [] Acceso a Confluence funcionando
- [] VPN corporativa instalada y probada
- [] 1Password corporativo configurado y contraseñas migradas
- [] 2FA activado en GitHub, Google Workspace y AWS Console
- [] Acceso a AWS Console (Read-only) confirmado
- [] Acceso a Datadog confirmado
- [] Acceso a SonarQube confirmado

12.2 Entorno de Desarrollo — Back-end

- [] SDKMAN instalado y Java 17 activo (`java -version`)
- [] IntelliJ IDEA instalado con plugins obligatorios
- [] Docker Desktop instalado y corriendo (`docker ps`)
- [] Repositorio clonado correctamente con SSH
- [] Archivo `.env` creado con todas las variables obligatorias
- [] `docker compose up -d` ejecutado sin errores
- [] Aplicación corriendo en localhost:8080 (`actuator/health` retorna UP)
- [] `./mvnw test` ejecutado sin fallos
- [] Swagger local accesible en localhost:8080/swagger-ui/index.html

12.3 Entorno de Desarrollo — Front-end

- [] NVM instalado y Node.js 20 activo (`node --version`)
- [] VS Code instalado con extensiones obligatorias
- [] Repositorio front-end clonado correctamente con SSH
- [] Archivo `.env` front-end creado con variables obligatorias
- [] `npm install` ejecutado sin errores
- [] `npm run dev` corriendo en localhost:5173
- [] `npm run test` ejecutado sin fallos

12.4 Flujo de Trabajo

- [] SSH Key configurada en GitHub
- [] Primer commit de prueba realizado siguiendo Conventional Commits
- [] Branch creada siguiendo el patrón de nomenclatura correcto
- [] Pull Request de prueba abierto con template completo
- [] Al menos 1 Code Review observado

- ☐ Daily del squad participado al menos 1 vez

12.5 Cultura y Conocimiento

- ☐ Guía de Ingeniería Back-end o Front-end leída completamente
- ☐ Sesión de bienvenida con Tech Lead realizada (mínimo 1h)
- ☐ Presentación al squad en la daily o reunión de equipo
- ☐ Al menos 1 sesión de pair programming con el buddy agendada
- ☐ Plan 30/60/90 días revisado y acordado con Tech Lead
- ☐ Página personal de onboarding creada en Confluence

12.6 Firma de Confirmación

| Rol | Nombre | Firma | Fecha |
|-------------------------|--------|-------|-------|
| Nuevo Desarrollador | | | |
| Tech Lead | | | |
| People Business Partner | | | |

SECCIÓN 13 — CONTACTOS ÚTILES

13.1 Tech Leads por Área

| Área | Nombre | Slack | Email |
|--------------------------|------------------------|---------------------|------------------------------|
| Tech Lead — Back-end | A confirmar por People | @tech-lead-backend | tl-backend@santopegasus.com |
| Tech Lead — Front-end | A confirmar por People | @tech-lead-frontend | tl-frontend@santopegasus.com |
| Tech Lead — DevOps / SRE | A confirmar por People | @tech-lead-devops | tl-devops@santopegasus.com |
| Dirección de Ingeniería | A confirmar por People | @eng-director | engineering@santopegasus.com |

> Los nombres específicos de los Tech Leads son comunicados en el correo de bienvenida enviado por People antes del Día 1.

13.2 Equipo de People & Talent

| Rol | Contacto | Canal |
|-------------------------|-------------------------|-----------------------|
| People Business Partner | people@santopegasus.com | `#people-hr` en Slack |
| Talent Acquisition | talent@santopegasus.com | `#people-hr` en Slack |

13.3 Equipo de IT / Soporte

| Canal | Propósito | Tiempo de Respuesta |
|------------------------------|--|---------------------|
| it-support@santopegasus.com | Solicitud de accesos, hardware, incidentes de IT | 4h hábiles |
| `#it-support` en Slack | Consultas rápidas de IT | 2h hábiles |
| Ticket en Jira (proyecto IT) | Solicitudes formales con seguimiento | Según prioridad |

13.4 Contactos de Emergencia

| Situación | A quién contactar | Cómo |
|--|------------------------------------|--|
| Incidente de producción | Tech Lead del squad + `#incidents` | Slack |
| Incidente de seguridad | Tech Lead de DevOps + IT/Soporte | Slack + Email |
| Problema de accesos urgente | IT/Soporte | Email + Slack |
| Situación personal (salud, emergencia) | People BP | Email o teléfono directo (comunicado en Día 1) |

SECCIÓN 14 — PREGUNTAS FRECUENTES DEL ONBOARDING

P: ¿Cuánto tiempo tengo para hacer el onboarding antes de que se espere que sea plenamente productivo?

R: El plan 30/60/90 días es una guía de rampaje progresiva. No se espera que seas plenamente productivo al finalizar la Semana 1. El objetivo de la primera semana es tener el entorno configurado y conocer el equipo. La productividad plena se espera aproximadamente al mes 3, según tu nivel de seniority.

P: ¿Puedo hacer preguntas "básicas" sin sentirme mal?

R: Absolutamente sí. Tenés un buddy asignado exactamente para eso. En Santo Pegasus, preguntar es una señal de inteligencia, no de ignorancia. Si una pregunta te parece básica, es probable que también le sirva a otra persona del equipo, así que preguntala en el canal correspondiente en Slack.

P: ¿Qué pasa si rompo algo en el ambiente local?

R: El ambiente local es exactamente para eso: experimentar, romper y aprender. Nada en tu entorno local afecta producción. Si tenés dudas, tu buddy puede ayudarte a reconstruirlo desde cero con ``docker compose down -v && docker compose up -d``.

P: ¿Puedo pushear código directamente a `develop` o `main`?

R: No. Las ramas ``develop`` y ``main`` están protegidas y solo reciben código a través de Pull Requests aprobados. Cualquier intento de push directo será rechazado por GitHub.

P: ¿En qué nivel de inglés debo estar para trabajar aquí?

R: La comunicación interna del equipo es principalmente en español. La documentación técnica (Guías de Ingeniería, ADRs) puede estar en inglés o español según quién la escribió. El código, los comentarios, los commits y los nombres de variables deben estar en inglés. Un nivel de lectura y escritura técnica básica es suficiente para comenzar.

P: ¿Cuándo puedo solicitar el presupuesto de cursos y certificaciones?

R: El presupuesto está disponible desde el primer mes. Sin embargo, recomendamos que en los primeros 90 días te enfoques en el onboarding y el aprendizaje del contexto del producto. Después de ese período, conversá con tu Tech Lead y People sobre las certificaciones más alineadas con tu plan de carrera.

P: ¿Cómo sé si mi Pull Request fue revisado?

R: Recibirás una notificación de GitHub por correo electrónico y también en el canal ``#code-reviews`` de Slack. También podés configurar las notificaciones de GitHub directamente en Slack usando la integración de GitHub disponible en el workspace.

P: ¿Qué es un ADR y cómo lo creo?

R: ADR significa Architecture Decision Record. Es un documento breve que registra una decisión técnica importante: el contexto, las opciones consideradas, la decisión tomada y sus consecuencias. El template está disponible en Confluence: `Engineering → Decisiones de Arquitectura → Template ADR`. Pedile a tu Tech Lead que revise el primero que escribas.

P: ¿Puedo usar `System.out.println()` para debug?

R: No en código que va al repositorio. La utilización de `System.out.println()` está estrictamente prohibida en Santo Pegasus. Usamos SLF4J como interfaz de abstracción con Logback como framework de implementación. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 9]

P: ¿Cómo reporto un bug que encontré en producción?

R: Abrí un ticket en Jira del tipo Bug en el proyecto del squad correspondiente. Si el bug es crítico y está afectando usuarios en producción ahora mismo, notificará primero en `#incidents` en Slack con una descripción breve y luego creá el ticket.

SECCIÓN 15 — DISPOSICIONES FINALES Y REVISIÓN DEL DOCUMENTO

15.1 Vigencia del Documento

Este manual entra en vigencia a partir de su fecha de publicación (Junio 2026) y aplica a todos los desarrolladores que se incorporen a Santo Pegasus Soluciones a partir de esa fecha.

15.2 Responsabilidades

| Rol | Responsabilidad |
|---------------------------------|---|
| People & Talent | Distribuir este manual a cada nuevo integrante antes del Día 1. Mantener la Sección 10 actualizada. |
| Tech Leads (todos los Chapters) | Asegurar que el checklist de la Sección 12 sea completado en la primera semana. Asignar un buddy a cada nuevo integrante. |
| Dirección de Ingeniería | Aprobar las revisiones del documento. Mantener la Sección 1 (Cultura) y Sección 2 (Estructura) alineadas con la realidad de la empresa. |
| IT / Soporte | Garantizar que los accesos listados en la Sección 3 estén disponibles en los plazos indicados. |

15.3 Ciclo de Revisión

Este documento es revisado formalmente cada 6 meses (próxima revisión: Diciembre 2026). Las actualizaciones pueden ocurrir de forma extraordinaria cuando:

- Hay cambios significativos en el stack tecnológico.
- Se incorporan o eliminan herramientas del flujo de trabajo.
- Hay cambios en la estructura del equipo o en los beneficios de RRHH.
- Se reciben sugerencias fundamentadas de miembros del equipo.

15.4 Cómo Contribuir a Este Documento

Este manual es un documento vivo. Si durante tu onboarding identificás información desactualizada, un paso que falta o una mejora posible:

1. Hacé un comentario directamente en la página de Confluence.
2. O enviá tu sugerencia al canal `#onboarding-feedback` en Slack.
3. O abrí un ticket en Jira del tipo Task en el proyecto People con el label `docs-improvement`.

Las contribuciones de nuevos integrantes son especialmente valiosas porque vienen de la perspectiva de alguien que acaba de pasar por el proceso. Tu mirada fresca importa.

15.5 Agradecimiento Final

Construir software de calidad es un deporte de equipo. Este manual representa el esfuerzo colectivo de todos los que pasaron por Santo Pegasus antes que vos, documentando lo que les habría gustado saber en su primer día.

Ahora sos parte de ese equipo. En algún momento, quizás seas vos quien ayude a escribir la próxima versión de este documento para darle la bienvenida a alguien nuevo.

¡Bienvenido a Santo Pegasus Soluciones. El código que escribas hoy importa.

> Versión: 1.0.0 | Última Actualización: Junio 2026 | Próxima Revisión: Diciembre 2026

> Departamento Responsable: People & Engineering — Santo Pegasus Soluciones

> Clasificación: Interno — Confidencial

> Aprobado por: Dirección de Ingeniería y People Business Partner

Este documento fue elaborado con base en la Guía Oficial de Ingeniería Back-end de Santo Pegasus Soluciones, Versión 2.4.0, Octubre 2025, Departamento de Ingeniería de Software / Chapter de Back-end.